

False Position Root Solver in MATLAB

Numerical Methods Portfolio Project

Author

Ehsan Shakil Ahmad

Developed Using

MATLAB Online

Project Date

June 2026

Keywords: Numerical Methods, MATLAB, Root Finding, False Position Method, Regula Falsi Method, Engineering Computation, Numerical Analysis

TABLE OF CONTENTS

1	Objectives	3
2	Theory	3
3	Nomenclature Table	4
4	Code Improvement	5
5	Flowchart	6
6	MATLAB Code	7
7	Output.....	8
8	Comparison with Bisection Method	9
9	Results	10
10	Conclusion.....	10

1 PROJECT OVERVIEW

This project presents a MATLAB implementation of the False Position (Regula Falsi) Method for solving nonlinear equations. The program automatically identifies a valid interval containing the root, performs iterative calculations until convergence, and visualizes the result graphically. The obtained solution is also compared with the Bisection Method to evaluate convergence performance and computational efficiency.

2 OBJECTIVES

The objectives of this project are:

- To understand the working principle of the False Position (Regula Falsi) Method.
- To implement the False Position Method in MATLAB for solving nonlinear equations.
- To automatically detect a valid interval containing the root.
- To determine the root of the equation:

$$f(x) = x^3 - x - 2$$

- To visualize the root graphically using MATLAB plots.
- To compare the performance of the False Position Method with the Bisection Method.

3 PROBLEM STATEMENT

Determine the root of the nonlinear equation:

$$f(x) = x^3 - x - 2$$

using the False Position (Regula Falsi) Method with a convergence tolerance of:

$$10^{-6}$$

and compare the results with those obtained using the Bisection Method.

4 THEORY

The False Position Method (Regula Falsi) is a bracketing numerical technique used to find the root of a non-linear equation. Similar to the Bisection Method, it requires an interval $[a, b]$ where the function changes sign:

$$f(a) \cdot f(b) < 0$$

Unlike the Bisection Method, which uses the midpoint of the interval, the False Position Method estimates the root using a straight line connecting the points $(a, f(a))$ and $(b, f(b))$.

The new approximation is calculated using:

$$c = \frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)}$$

The interval is then updated while preserving the sign change condition until the desired accuracy is achieved.

This method generally converges faster than the Bisection Method because it uses information about the function values rather than relying solely on interval halving.

4.1 ADVANTAGES

- Simple and easy to implement.
- Guarantees convergence when a valid bracketing interval exists.
- Generally converges faster than the Bisection Method.
- Does not require derivative evaluation.

4.2 LIMITATIONS

- Convergence may become slow when one endpoint remains fixed for many iterations.
- Requires an initial interval with a sign change.
- Usually slower than derivative-based methods such as Newton-Raphson.

5 NOMENCLATURE TABLE

Symbol	Description
$f(x)$	Nonlinear function
a	Lower interval limit
b	Upper interval limit
c	Estimated root
tol	Convergence tolerance
$iter$	Iteration counter
$f(a)$	Function value at lower limit
$f(b)$	Function value at upper limit
$f(c)$	Function value at estimated root

6 ALGORITHM

The False Position Method follows the steps below to determine the root of a nonlinear equation:

1. Define the nonlinear function $f(x)$ and specify the search range, step size, and convergence tolerance.
2. Search the specified range for two consecutive points where the function changes sign, satisfying:

$$f(a)f(b) < 0$$

3. Select the detected interval $[a, b]$ as the initial bracketing interval containing the root.
4. Compute a new approximation of the root using the False Position formula:

$$c = \frac{af(b) - bf(a)}{f(b) - f(a)}$$

5. Display the current iteration number and the values of a , b , and c .
6. Check the convergence criterion:

$$|f(c)| < \text{tolerance}$$

If satisfied, terminate the iteration and accept c as the root.

7. If convergence is not achieved, determine the new interval:
 - If $f(a)f(c) < 0$, set $b = c$.
 - Otherwise, set $a = c$.
8. Repeat Steps 4–7 until the convergence criterion is satisfied.
9. Display the final root and total number of iterations.
10. Plot the function and mark the computed root on the graph for visualization.

7 CODE IMPROVEMENTS

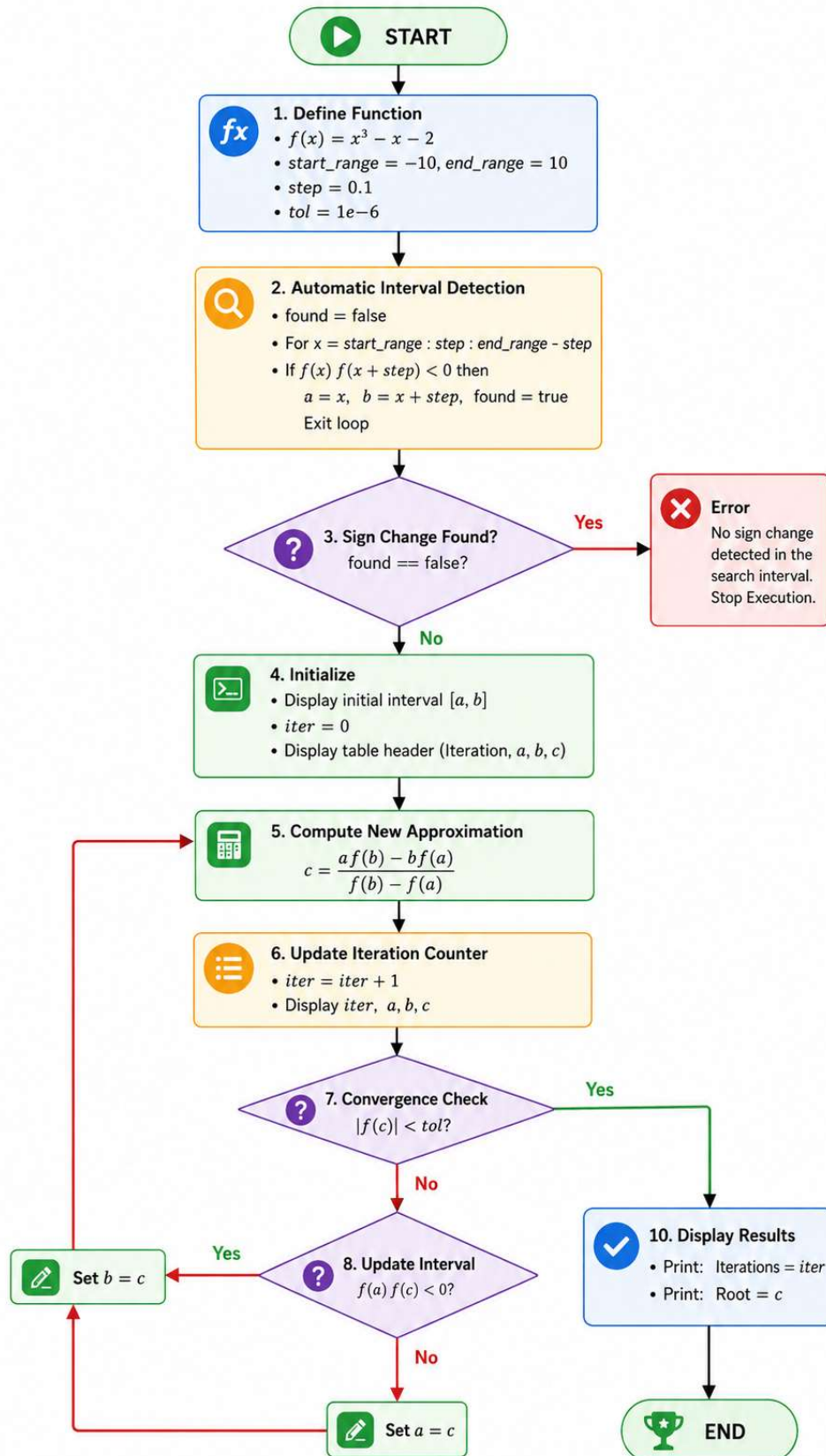
Several improvements were incorporated into the MATLAB implementation:

- Automatic interval detection over a user-defined search range.
- Error handling when no valid interval is found.
- Iteration tracking and formatted output display.
- Convergence checking using a specified tolerance.
- Root visualization through MATLAB plotting.
- Clean and structured code for readability and future modifications.

These enhancements make the solver more reliable and user-friendly compared to a basic implementation.

8 FLOWCHART

False Position (Regula Falsi) Method Flowchart



9 MATLAB CODE

```
% Define function
f = @(x) x.^3 - x - 2;

% Search range and step size
start_range = -10;
end_range = 10;
step = 0.1;

% Tolerance
tol = 1e-6;

% Automatic interval detection
found = false;

for x = start_range:step:end_range-step
    if f(x)*f(x+step) < 0
        a = x;
        b = x + step;
        found = true;
        break
    end
end

if ~found
    error('No sign change detected in the search interval.')
end

fprintf('Initial interval found: [%.4f, %.4f]\n', a, b)

% Iteration counter
iter = 0;

fprintf('Iteration\t a\t\t b\t\t c\n')

while true

    c = (a*f(b) - b*f(a))/(f(b) - f(a));

    iter = iter + 1;

    fprintf('%d\t\t %.6f\t %.6f\t %.6f\n', iter, a, b, c)

    if abs(f(c)) < tol
        break
    end

    if f(a)*f(c) < 0
        b = c;
    else
        a = c;
    end

end

fprintf('Iterations = %d\n', iter)
fprintf('Root = %.6f\n', c)

% Plotting
```

```

x = 0:0.01:3;

figure
plot(x, f(x), 'LineWidth', 2)
grid on
hold on

plot(c, f(c), 'ro', 'MarkerSize', 10)

xlabel('x')
ylabel('f(x)')
title('False Position Method Root')

```

10 OUTPUT

```

Initial interval found: [1.5000, 1.6000]
Iteration      a          b          c
1             1.500000    1.600000    1.520129
2             1.520129    1.600000    1.521307
3             1.521307    1.600000    1.521376
4             1.521376    1.600000    1.521379
5             1.521379    1.600000    1.521380
Iterations = 5
Root = 1.521380

```

10.1 FINAL NUMERICAL SOLUTION

Parameter	Value
Root	1.521380
Iterations	5
Tolerance	10 ⁻⁶

11 CONVERGENCE TABLE

Table 1. Convergence history of the False Position Method.

Iteration	a	b	c	f(c)
1	1.500000	1.600000	1.520129	-0.007550
2	1.520129	1.600000	1.521307	-0.000430
3	1.521307	1.600000	1.521376	-0.000025
4	1.521376	1.600000	1.521379	-0.000001
5	1.521379	1.600000	1.521380	≈ 0

12 GRAPHICAL OUTPUT

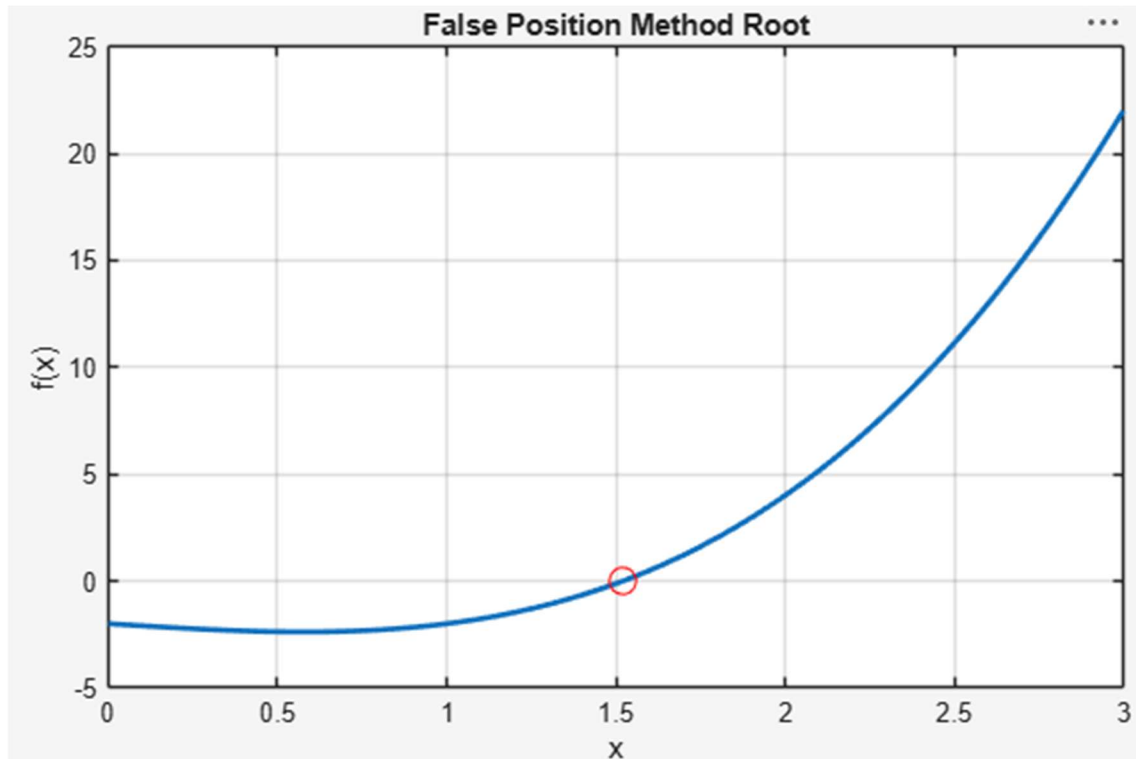


Figure 1. Root obtained using the False Position Method for $f(x)$

13 COMPARISON WITH BISECTION METHOD

Both the Bisection Method and False Position Method are bracketing techniques that require a sign change within the interval. However, their approaches differ.

The Bisection Method repeatedly halves the interval, while the False Position Method estimates the root using a secant line between the interval endpoints. As a result, the False Position Method often reaches the solution in fewer iterations.

13.1 PERFORMANCE COMPARISON

Method	Root	Iterations	Requires Derivative
Bisection	1.521380	20	No
False Position	1.521380	5	No

The False Position Method achieved the same root with significantly fewer iterations, demonstrating improved computational efficiency for this problem.

14 APPLICATIONS

The False Position Method is commonly used in:

- Mechanical Engineering calculations
- Structural analysis
- Fluid mechanics problems
- Heat transfer equations
- Nonlinear engineering models
- Scientific computing

This section makes the report feel more engineering-oriented.

15 RESULTS

The MATLAB implementation successfully determined the root of the nonlinear equation:

$$x^3 - x - 2 = 0$$

The computed root was:

$$x = 1.521380$$

The method converged within 5 iterations while maintaining the root inside a valid bracketing interval. The graphical output confirmed that the computed value corresponds to the point where the function crosses the x-axis.

16 DISCUSSION

The False Position Method successfully located the root of the equation $x^3 - x - 2 = 0$ within five iterations. The convergence table shows that the approximation rapidly approached the actual root value, while the magnitude of $f(c)$ decreased toward zero. Compared with the Bisection Method, which required twenty iterations for the same tolerance, the False Position Method demonstrated superior efficiency by utilizing function values to estimate the root location rather than repeatedly halving the interval.

17 CONCLUSION

A MATLAB solver based on the False Position Method was successfully developed and tested. The program automatically identified a suitable interval, iteratively approximated the root, and displayed the results numerically and graphically.

For the given equation, the method converged to the root:

$$x = 1.521380$$

in only 5 iterations. When compared with the Bisection Method, the False Position Method demonstrated faster convergence while retaining the robustness of interval-based root-finding techniques.

This project strengthened understanding of numerical root-finding methods, MATLAB programming, convergence behavior, and engineering computation techniques.

18 FUTURE IMPROVEMENTS

Future versions of this solver may include:

- User-defined functions through keyboard input.
- Multiple-root detection.
- Graphical user interface (GUI) implementation.
- Automatic comparison with other numerical methods.
- Export of iteration data to Excel files.